

Project Plan

Campus Marketplace - UniSquare

COMS3009A: Software Design III

Name of Group: Dev-astators

Team:

Name	Role	Primary Responsibilities
Nkosinathi Khumalo	Full-stack	Auth services, middleware, listing management, SA data integration, facility staff management, testing
Kamogelo Mosehle	Full-stack	CI/CD pipeline setup, Category filtering, Payment integration, Booking & slot management, Frontend testing
Mpho Murashiwa	Frontend	Page Designs and Improvements, UML Diagrams, Weekly Documentation, TrelloTaskManagement
Makubung Tjeding	Backend	Writing RLS policies in supabase, creating listings, sign in and sign out on dashboards, assigning roles on admin page.
Karabo Tlaka	Frontend	Designed Staff-facility page, Admin exportion csv/pdf,Sign-in Page,Admin Analytics
Karabo Kutumela	Backend	Messaging system,Notifications,database setup in Supabase,defining database schema,RLS policies,Ratings and Reviews,SQL queries,,front-end testing,Integrating Supabase on Github

1. Project Overview

UniSquare is a web-based peer-to-peer campus marketplace built exclusively for Wits University students. It enables students to list, discover, and safely exchange or trade new and used items within the campus community. All transactions are processed through a secure trade facility where items are physically handed over, with payments handled through PayFast.

The platform is built across four agile sprints over approximately seven weeks, following Scrum methodology with CI/CD pipelines, test-driven development, and a South African data integration requirement using real Stats SA Consumer Price Index data.

2. Project Objectives

The main objectives of the project were:

- Provide a secure marketplace for students.
- Enable safe item exchanges via campus facilities.
- Support hybrid payments (cash + online).
- Online payments can have a cash shortfall supplement.
- Improve trust through ratings and reviews.
- Deliver a responsive and scalable system.
- Maintain at least 80% code coverage by the end of the project
- Integrate real South African CPI data (Stats SA P0141) for listing price suggestions.
- Follow scrum methodology with documented sprints, retrospectives and CI/CD pipeline.

3. Scope:

The following features are in scope for the final product:

- Google OAuth sign-in restricted to Wits University email domains
- Note: For testing and submission, we disabled the email restriction.
- Three user roles: Student, Facility Staff, Admin, each with a dedicated dashboard
 - Listing creation, editing, deletion with image upload and Stats SA price suggestions
 - In-app messaging between buyers and sellers
 - PayFast payment integration with partial payment and cash shortfall tracking
 - Trade facility booking system (drop-off and collection slots)
 - Rating and review system with profile aggregation
 - In-app notifications for transactions, messages and bookings
 - Admin analytics dashboard with CSV export

4. Tech Stack

1. **Frontend:** ReactJS + Vite + Tailwind CSS
2. **Backend:** NodeJS + ExpressJS
3. **Database:** Supabase(PostgreSQL)
4. **Authentication:** Google OAuth
5. **Payment gateway:** PayFast
6. **Hosting:** Azure Static Web App + Azure App Service (client and server deployed separately)
7. **CI/CD:** GitHub Actions
8. **Testing & Code coverage:** Jest & Codecov

As a team, we chose to use the above stack for rapid development. We chose React for its component-based development, which allows us to easily reuse UI elements such as buttons, forms and cards. Tailwind CSS removes the need for us to write custom CSS. We chose to use Supabase for the database as it provides ready-to-use authentication, eliminates the need for us to build DB APIs and allows for greater scalability of the platform.

5. Sprint Plan and Milestones

The project follows a Scrum methodology across all four sprints. The sprint durations vary based on complexity and assessment scheduling.

Sprint	Start Date	End Date	Duration	Focus	Status
Sprint 1	30 March 2026	13 April 2026	2 weeks	Authentication, onboarding, role-based routing, student dashboard	Complete
Sprint 2	14 April 2026	20 April 2026	1 week	Listing management, messaging, buyer experience	Complete
Sprint 3	21 April 2026	11 May 2026	2 weeks	Trade facility management, payment integration, SA data integration frontend	Complete
Sprint 4	12 May 2026	18 May 2026	1 week	Ratings & reviews, notifications, analytics dashboard	Complete

5.1 Sprint 1- Authentication and Onboarding (2 weeks)

30 March 2026 - 13 April 2026

- Set up project repository, GitHub actions, CI/CD pipeline and Azure deployment
- Implement Google OAuth+ Supabase Auth sign-in and log-ins.
- Building and designing the student dashboard.
- Setting up Jest with code coverage reporting.

5.2 Sprint 2 - Listings and Messaging (2 weeks)

14 April 2026 - 20 April 2026

- Implement and design student (buyer/seller) dashboard
- Implementing listing creation with image uploads to Supabase storage.
- Add edit and delete functionality for student listings.
- Build listing details page.
- Implement in-app messaging between buyers and sellers scoped to listings.
- Write unit tests for the functionality of the features.

5.3 Sprint 3 - Trade Facility and Payments (3 Weeks)

21 April 2026 - 11 May 2026

- Build facility and Admin dashboards
- Facility staff dashboard shows daily booking schedule and transaction progress
- Build trade facility booking system for drop-off and collection slots with capacity enforcements.
- Build admin facility configuration (operating hours, slot capacity)
- Integrate PayFast payment gateway with partial shortfall tracking,
- Integrate Stats SA CPI data for suggested price ranges on the create listing form

5.3 Sprint Ratings, Notifications and Analytics (1 Week)

12 May 2026 - 18 May 2026

- Implement a rating and review system triggered after completed transactions
- Build profile rating aggregation showing average score and total reviews
- Implement in-app notifications for payments, bookings and messages using Supabase Realtime
- Build admin analytics dashboard with popular categories, transaction volume and CSV/PDF export
- Final polish: resolving bugs, cleaning up code.
- Achieve $\geq 80\%$ target for code coverage

6. SA Data Integration

The platform satisfies the South African data integration requirement by consuming CPI data published by Statistics South Africa (StatsSA) in their monthly P0141 Consumer Price Index release.

6.1 Data Source

- Source: Statistics South Africa — Consumer Price Index (CPI), March 2026 release (P0141)
- URL: https://www.statssa.gov.za/?page_id=1854&PPN=P0141
- Base year: December 2024 = 100
- Headline annual inflation: 3.1% (March 2026 vs March 2025)

6.2 Category Mapping

Platform Category	Stats SA Category (Table E)	CPI Index	Annual Change	Justification
Textbooks	Education services — Tertiary	108.1	+4.2%	University fees surveyed annually in March
Electronics	Information & communication equipment	88.2	-9.6%	Reflects falling tech prices trend
Furniture	Furniture, furnishings & loose carpets	95.9	-5.0%	Household furniture category
Clothing	Clothing and footwear	101.6	+1.2%	Directly matches listing category
Other	Personal care & miscellaneous services	103.9	+2.8%	Catch-all for uncategorised items

6.3 Implementation

- CPI data is stored in `server/src/data/cpiData.json` and bundled with the server at build time
- The `cpiService.js` service reads this file and calculates a suggested price range using the annual percentage change for the listing category
- The suggested price range appears on the `CreateListing` form as soon as both category and asking price are filled in
- The frontend displays the Stats SA source, reference date, CPI index and annual change alongside the suggested range
- The choice to bundle the data rather than call Stats SA at runtime was deliberate as Stats SA does not provide a stable REST API, and bundling ensures the feature never fails due to an external dependency

7. System Architecture Overview

The system follows a three-tier architecture with a React frontend, Express backend API and Supabase as the data and auth layer

7.1 System Components

- Client – React + Vite SPA deployed to Azure Static Web App. Communicates with the backend via HTTPS REST calls with Supabase JWT tokens attached to every request
- Server – Node.js + Express API deployed to Azure App Service. A layered middleware chain protects every route.
- Supabase – Provides PostgreSQL database with Row Level Security, Google OAuth, file storage for listing images, and Realtime subscriptions for notifications
- PayFast – Third-party payment gateway handling the online portion of transactions. The backend receives ITN webhook callbacks to confirm payments.

7.2 UML Diagrams

The following UML diagrams are included in the project artefacts:

- Use Case Diagram – all user stories across 4 sprints mapped to 3 actors
- Class Diagram – domain models, services and middleware classes with relationships
- Component Diagram – package-level dependencies across client, server and external systems
- Sequence Diagram – listing creation flow showing the full middleware chain
- Deployment Diagram – physical deployment across developer machine, GitHub, Azure and Supabase

8. CI/CD Pipeline

GitHub Actions is configured to run every pull request to main. The pipeline consists of 2 parallel jobs.

- Client job – install dependencies, run ESLint, build the Vite bundle (VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY injected from GitHub Secrets)
- Server job – install dependencies, run ESLint, run all Jest tests with coverage reporting, upload coverage report as a build artefact
- On merge to main – GitHub Actions deploys the client to Azure Static Web App and the server to Azure App Service
- Releases are tagged per sprint in the GitHub repository